

Sausage Man: The Dungeon Crusade

1. Project Overview

Sausage Man: The Dungeon Crusade is a top-down roguelite action game inspired by the fast-paced dungeon-crawling gameplay of Soul Knight, combined with the deep character progression and equipment system of Ragnarok Online. Players control a customizable Sausage Man character who fights through procedurally generated dungeon stages filled with diverse bot enemies, collecting weapons, armor, and accessories to grow stronger. The game features:

- Multiple dungeon stages with escalating difficulty and distinct environments
- Diverse enemy bots with unique AI behaviors (patrol, chase, ranged attack, boss)
- A rich equipment system with weapons, armor, and accessories across multiple item grades
- A character leveling system inspired by Ragnarok Online (EXP, stats, skill points)
- Real-time statistical data collection for gameplay analysis and visualization

2. Project Review

SoulKnight (ChillyRoom,2017) is a widely popular mobile dungeon-crawler with simple controls, procedurally generated levels, many weapons, and co-op support. It serves as the primary gameplay inspiration for Sausage Man: The Dungeon Crusade.

- Key Improvements Over Existing Projects
Equipment & Grade System: Unlike Soul Knight's weapon-only focus, our game introduces full RPG-style equipment (weapon, armor, accessories) with grade tiers (Normal, Rare, Epic, Legendary) similar to Ragnarok Online.
Character Leveling & Stats: Players gain EXP from defeating enemies to level up, distributing stat points (STR, VIT, AGI, INT) that directly affect combat performance.
Richer Statistical Collection: We collect and visualize at

minimum 5 gameplay features per session, providing actionable insights into player behavior and game balance. Sausage Man Theme: A unique visual identity with a humorous sausage-character aesthetic that appeals broadly to casual and core gamers alike.

3. Programming Development

3.1 Game Concept

Sausage Man: The Dungeon Crusade is a 2D top-down roguelite dungeon crawler. The player selects a Sausage Man character and descends through dungeon stages. Each stage is populated with multiple rooms. Enemies patrol and attack; clearing all enemies unlocks the next room or stage. At the end of each stage, a Boss enemy must be defeated to progress. Between stages, players visit a Shop Room to buy or upgrade equipment using Gold dropped by enemies.

Core mechanics include:

- Movement: WASD or arrow keys; mouse or auto-aim for attack direction
- Combat: Melee and ranged weapon types; skill cooldowns tied to character class
- Equipment: Equip up to 4 slots (Weapon, Armor, Helmet, Accessory); each item has grade-based stat bonuses
- Leveling: Defeat enemies to gain EXP; level up to earn Stat Points and Skill Points
- Stage Progression: Stages 1–10 in 3 dungeon environments (Forest, Cave, Castle); each has unique tilesets and enemy sets

3.2 Object-Oriented Programming Implementation

The game is built entirely in Python using Pygame, following strict OOP principles. The following classes are planned:

Class	Role	Key Attributes & Methods
Character	Base class for all characters	hp, maxHp, atk, def, spd; move(), takeDamage(), die()
Player	Inherits Character; player-controlled	level, exp, statPoints, equipment[]; levelUp(), equipItem(), useSkill()
Enemy	Inherits Character; bot-controlled	enemyType, expReward, goldDrop, aiState; updateAI(), patrol(), chaseTarget()
Boss	Inherits Enemy; multi-phase boss	phase, phaseThresholds, specialAttacks[]; triggerPhase(), specialAttack()
Equipment	Represents any equipable item	name, grade, slot, stats{}; getDescription(), applyStats()
Stage	Manages dungeon stage and rooms	stageId, rooms[], currentRoom; loadRooms(), nextRoom(), isClear()
Room	Single dungeon room logic	enemies[], items[], layout; spawnEnemies(), checkCleared()
StatsCollector	Records gameplay statistics	records[]; record(), exportCSV(), getSummary()

3.3 Algorithms Involved

Pathfinding (A* Algorithm): Enemy bots use A* pathfinding on the tile-based dungeon grid to navigate toward the player while avoiding walls and obstacles.

- AI State Machine: Each enemy operates on a Finite State Machine (FSM) with states: Idle, Patrol, Chase, Attack, Flee. Transitions are triggered by proximity and health thresholds.
- Procedural Room Layout: Dungeon rooms are generated using a random room template selection algorithm, placing enemies, items, and traps based on weighted probability tables.
- Loot Drop System (Weighted Random): Equipment drops are resolved using a weighted random algorithm based on enemy type and stage, with grade probabilities (Normal: 60%, Rare: 25%, Epic: 12%, Legendary: 3%).
- Collision Detection: AABB (Axis-Aligned Bounding Box) collision is used for player-enemy, projectile-character, and character-wall interactions.
- Stat Formula: Player combat stats scale with level and equipment using a formula: $\text{Effective ATK} = \text{BaseATK} + (\text{STR} \times 1.5) + \text{WeaponATK} + \text{GradeBonus}$.

4. Statistical Data (Prop Stats)

4.1 Data Features

The following 5 measurable features will be tracked per gameplay session, with a target of 100+ data records per feature:

# Feature	Description	Example Values
1 Player Score	Total score accumulated per run, based on kills, items collected, and stage reached.	0 – 999,999 pts
2 Enemies Defeated	Total number of enemy bots defeated per run.	0 – 500+
3 Damage Dealt	Total damage dealt to all enemies per run.	0 – 50,000+
4 Time Played (seconds)	Total time spent in gameplay per run.	60 – 3600 sec

5	Stage Reached	Highest dungeon stage reached before player death.	1 – 10
6	Items Collected	Total equipment items collected per run (bonus feature).	0 – 50+
7	Deaths	Number of times the player died per session (bonus feature).	0 – 20

Data will be collected automatically during each gameplay session. A minimum of 100 game runs will be played (or simulated under controlled bot conditions) to meet the 100+ records requirement per feature.

4.2 Data Recording Method

All statistical data will be recorded using the StatsCollector class. At the end of each game run, the collector serializes the session record as a row appended to a local CSV file (gameplay_stats.csv). This approach is chosen because:

- CSV is human-readable and easily importable into Python (pandas), Excel, or Google Sheets for analysis.
- Each row represents one complete game run with columns for all tracked features plus metadata (timestamp, player name, character class).
- The file is saved to the user's local machine after each session, ensuring no data loss.

Sample CSV columns: run_id, timestamp, player_name, score, enemies_defeated, damage_dealt, time_played_sec, stage_reached, items_collected, deaths

4.3 Data Analysis Report

After sufficient data is collected, the following analyses will be performed using Python (pandas, matplotlib, seaborn):

- Descriptive Statistics: Mean, median, standard deviation, min/max for all numeric features.
- Score Distribution: Histogram of player scores to identify common score ranges and outliers.

- Stage Reached vs. Time Played: Scatter plot to analyze correlation between play duration and dungeon depth.
- Enemies Defeated Over Runs: Line chart tracking improvement in player performance across sessions.
- Damage Dealt Distribution: Box plot comparing damage dealt across different stage ranges.
- Heatmap: Correlation matrix of all numeric features to reveal interdependencies.

All charts will be embedded in the Final Project Report and displayed via an in-game Statistics Screen accessible from the main menu.

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Implement core classes: Character, Player, Enemy. Basic movement and collision.
3	10	Implement Stage, Room classes. Procedural room generation. Enemy spawning.
4	11	Implement Equipment system, leveling, stat formulas. Loot drop system.
5	12	Implement enemy AI (FSM + A* pathfinding). Boss class and boss fight logic.
6	13	Integrate StatsCollector. CSV export. Build statistics visualization screen.
7	14	Submission week (Draft)

6. Document version

Version: 1.0

Date: 4 March 2026

Editor: Inthat Niramarn 6810545999